



TALLER PRÁCTICO SEMANA 9: Persistencia con MySQL y Documentación de API REST

Unidad Curricular: Interfaces Web con el Usuario (INU-554)

Carrera: Informática - 5to Semestre

Docente: Ing. José Daniel Cadenas L.

Evaluación: Sumativa 10% (Individual)

Duración: 2 sesiones de clase (Martes 29/09: 3 horas de 45 min / Jueves 01/10: 2 horas académicas)

Fecha de entrega: Jueves 01/10/2026, al finalizar la sesión de clase



OBJETIVOS DE APRENDIZAJE

Al finalizar este taller, el estudiante será capaz de:

1. **Migrar la persistencia de datos** desde memoria RAM (arreglos en Node.js) hacia una base de datos relacional MySQL.
2. **Modelar entidades** del módulo de tesorería (contactos y movimientos caja/banco) mediante un Diagrama Entidad-Relación (DER).
3. **Documentar profesionalmente** una API REST con evidencia de 4 peticiones fundamentales: crear cliente, registrar movimiento, consultar datos y generar reporte.
4. **Interpretar respuestas del servidor** en formato JSON y sus códigos de estado HTTP.
5. **Validar la precisión** de los cálculos financieros del sistema comparándolos con cálculos manuales.
6. **Defender y modificar su código en vivo** (Prueba de Fuego), demostrando comprensión real de la arquitectura implementada.



CONTINUIDAD CON SEMANAS ANTERIORES

Semana	Logro alcanzado	Limitación detectada
Semana 7	Servidor Express con endpoints CRUD para contactos y movimientos	Datos en memoria RAM (se pierden al reiniciar)
Semana 8	Frontend Vue 3 consumiendo la API con Axios, async/await y try/catch	Los datos siguen sin persistir entre sesiones
Semana 9 (HOY)	Persistencia real con MySQL + Documentación profesional + Defensa en vivo	—



CONTENIDOS SEGÚN CONTRATO DE APRENDIZAJE

Este taller cubre los siguientes contenidos del **Bloque 3**:

- ☒ Backend: Arquitectura RESTful en Node.js, Express y persistencia con MySQL (CRUD).
- ☒ Modelado de entidades: contactos, movimientos caja banco.
- ☒ Seguridad: CORS, validaciones.

-  Operaciones CRUD aplicadas a la contabilidad de tesorería.
-  Interpretación de respuestas y códigos de estado. Formato JSON.
-  Configuración de peticiones en herramientas de testing (Postman/Insomnia).



REGLA DE ORO: "HUELLA DIGITAL" (OBLIGATORIO)

 **Para garantizar la originalidad de tu trabajo, debes personalizar tu base de datos y tus datos de prueba con tu identidad. Si no cumples esto, el trabajo se devuelve sin calificar.**

1. **Nombre de la Base de Datos:** Debe llamarse exactamente `erp_contable_[TUS_INICIALES]`.
Ejemplo: Si te llamas José Daniel Cadenas → `erp_contable_jdc`
2. **Datos de Prueba Únicos:** En tu script SQL de datos de prueba (`INSERT`), el primer contacto debe tener tu **Nombre y Apellido real**, y el primer movimiento debe tener en el concepto:
`"Ajuste inicial Cédula: [TÚ NÚMERO DE CÉDULA]"`
3. **Evidencia en el PDF:** En las capturas de Postman debe verse claramente tu nombre en el JSON de respuesta y tu cédula en el concepto del movimiento.



SESIÓN 1 — MARTES 29/09/2026 (3 horas de 45 min)



Objetivo de la sesión: Migrar a MySQL, crear el DER y levantar el servidor con base de datos real.

EJERCICIO 1: Instalación y configuración de MySQL (30 min)

Paso 1.1 — Verificar instalación de MySQL

Abre una terminal y ejecuta:

```
mysql --version
```

Si no está instalado, descarga el instalador desde <https://dev.mysql.com/downloads/installer/> o usa XAMPP/WAMP.

Paso 1.2 — Acceder al servidor MySQL

```
mysql -u root -p
```

Paso 1.3 — Crear la base de datos del ERP (con tu Huella Digital)

```
-- Reemplaza "jdc" por tus iniciales reales
CREATE DATABASE erp_contable_jdc CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE erp_contable_jdc;
```

Paso 1.4 — Crear usuario específico (buena práctica de seguridad)

```
CREATE USER 'erp_user'@'localhost' IDENTIFIED BY 'erp2026';
GRANT ALL PRIVILEGES ON erp_contable_jdc.* TO 'erp_user'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

✅ **Criterio de éxito:** Puedes acceder a la base de datos `erp_contable_[tus iniciales]` con el usuario `erp_user`.

EJERCICIO 2: Modelado de Entidades — Diagrama Entidad-Relación (45 min)

Según el contrato de aprendizaje, el modelado se limita a las entidades de **tesorería**: `contactos` y `movimientos` caja/banco.

Paso 2.1 — Identifica los atributos de cada entidad

Entidad: `contactos`

Atributo	Tipo de dato	Restricción
id	INT	PRIMARY KEY, AUTO_INCREMENT
nombre	VARCHAR(100)	NOT NULL
rfc	VARCHAR(13)	UNIQUE, NOT NULL
tipo	ENUM('Cliente','Proveedor')	NOT NULL
email	VARCHAR(100)	—
telefono	VARCHAR(20)	—
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Entidad: `movimientos`

Atributo	Tipo de dato	Restricción
id	INT	PRIMARY KEY, AUTO_INCREMENT
concepto	VARCHAR(200)	NOT NULL
tipo	ENUM('Ingreso','Egreso')	NOT NULL
monto	DECIMAL(10,2)	NOT NULL, CHECK (monto > 0)
fecha	DATE	NOT NULL

Atributo	Tipo de dato	Restricción
contacto_id	INT	FOREIGN KEY → contactos(id), ON DELETE SET NULL
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Paso 2.2 — Crea el Diagrama Entidad-Relación (DER)

Usa una de estas herramientas:

- **draw.io** (<https://app.diagrams.net/>) — gratuito y online
- **Lucidchart** (<https://www.lucidchart.com/>)
- **MySQL Workbench**

Elementos obligatorios del DER:

- Rectángulo por cada entidad con sus atributos
- Llaves primarias (PK) marcadas
- Llaves foráneas (FK) marcadas
- Línea de relación con cardinalidad **1:N** (un contacto puede tener muchos movimientos)
- Indicación de la restricción **ON DELETE SET NULL**

Paso 2.3 — Exporta el DER

Guarda el diagrama como imagen **PNG o JPG** con nombre: **DER_erp_contable.png**

✅ **Criterio de éxito:** Tienes un DER claro que muestra la relación 1:N entre Contactos y Movimientos.

● EJERCICIO 3: Creación de tablas en MySQL (30 min)

Paso 3.1 — Ejecuta el script SQL

💡 **IMPORTANTE:** Reemplaza **erp_contable_jdc** por el nombre de tu base de datos (con tus iniciales) y personaliza los datos de prueba con tu nombre y cédula (Huella Digital).

```
USE erp_contable_jdc;

-- =====
-- TABLA DE CONTACTOS
-- =====
-- Almacena clientes y proveedores del ERP
CREATE TABLE contactos (
    id INT AUTO_INCREMENT PRIMARY KEY,           -- Identificador único
    nombre VARCHAR(100) NOT NULL,                 -- Nombre del contacto
    rfc VARCHAR(13) UNIQUE NOT NULL,              -- RFC único para evitar duplicados
    tipo ENUM('Cliente', 'Proveedor') NOT NULL,   -- Solo permite 'Cliente' o
    email VARCHAR(100),                           -- Correo electrónico (opcional)
    telefono VARCHAR(20),                          -- Teléfono (opcional)
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- Fecha automática de
```

```

creación
);

-- =====
-- TABLA DE MOVIMIENTOS (CAJA/BANCO)
-- =====
-- Registra ingresos y egresos de tesorería
CREATE TABLE movimientos (
    id INT AUTO_INCREMENT PRIMARY KEY,          -- Identificador único
    concepto VARCHAR(200) NOT NULL,             -- Descripción del movimiento
    tipo ENUM('Ingreso', 'Egreso') NOT NULL,     -- Solo permite 'Ingreso' o
    'Egreso'
    monto DECIMAL(10,2) NOT NULL CHECK (monto > 0), -- Monto con 2 decimales,
    siempre positivo
    fecha DATE NOT NULL,                         -- Fecha del movimiento
    (obligatoria)
    contacto_id INT,                             -- FK: referencia al contacto
    asociado
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- Fecha automática de
    creación
    -- Relación con la tabla contactos:
    -- Si se elimina un contacto, sus movimientos NO se borran, solo queda
    contacto_id = NULL
    FOREIGN KEY (contacto_id) REFERENCES contactos(id) ON DELETE SET NULL
);

-- =====
-- DATOS DE PRUEBA (PERSONALIZAR CON TU HUELLA DIGITAL)
-- =====
INSERT INTO contactos (nombre, rfc, tipo, email) VALUES
('Tu Nombre y Apellido Real', 'TU123456RFC', 'Cliente', 'tu@email.com'),
('Proveedor Global', 'PGL987654XYZ', 'Proveedor', 'ventas@global.com');

INSERT INTO movimientos (concepto, tipo, monto, fecha, contacto_id) VALUES
('Ajuste inicial Cédula: V-12345678', 'Ingreso', 1500.00, '2026-09-15', 1),
('Compra de insumos', 'Egreso', 450.50, '2026-09-16', 2);

```

Paso 3.2 — Verifica la creación

```

SHOW TABLES;
DESCRIBE contactos;
DESCRIBE movimientos;
SELECT * FROM contactos;
SELECT * FROM movimientos;

```

✅ **Criterio de éxito:** Las tablas existen, tienen la estructura correcta y contienen los datos de prueba con tu nombre y cédula.

EJERCICIO 4: Modificación del servidor Express para usar MySQL (60 min)

Paso 4.1 — Instala el paquete MySQL para Node.js

```
cd mi-servidor
npm install mysql2
```

Paso 4.2 — Crea el archivo de configuración de BD

Crea el archivo `mi-servidor/config/db.js` con el siguiente código **comentado**:

```
// =====
// CONFIGURACIÓN DE CONEXIÓN A MySQL
// =====
// Este archivo centraliza la conexión a la base de datos
// para que todos los endpoints puedan reutilizarla.

// Importamos el paquete mysql2 (ya instalado con npm install mysql2)
const mysql = require('mysql2');

// Creamos un "pool" de conexiones.
// Un pool mantiene varias conexiones abiertas y las reutiliza,
// lo cual es más eficiente que abrir y cerrar una conexión en cada petición.
const pool = mysql.createPool({
  host: 'localhost',           // Servidor donde está MySQL (local en este caso)
  user: 'erp_user',           // Usuario creado en el Paso 1.4
  password: 'erp2026',        // Contraseña del usuario
  database: 'erp_contable_jdc', // ⚠ REEMPLAZA con el nombre de TU base de datos
                                // (erp_contable_[tus iniciales])
  waitForConnections: true,    // Si no hay conexiones libres, espera en cola
  connectionLimit: 10,        // Máximo de conexiones simultáneas
  queueLimit: 0                // Sin límite de peticiones en cola
});

// Exportamos el pool como "promesas" para poder usar async/await
// en los endpoints en lugar de callbacks.
module.exports = pool.promise();
```

Paso 4.3 — Reemplaza `server.js` con la versión MySQL

Reemplaza el contenido actual de `mi-servidor/server.js` por el siguiente código. **Observa los comentarios que explican cada parte**:

```
// =====
// SERVIDOR EXPRESS CON MySQL - ERP CONTABLE
// =====
// Este archivo define todos los endpoints (rutas) de la API REST
// y los conecta a la base de datos MySQL en lugar de usar arreglos en memoria.

// Importamos las dependencias necesarias
const express = require('express'); // Framework web para Node.js
const cors = require('cors');        // Middleware para permitir peticiones desde
```

```

otros orígenes (Vue en puerto 5173)
const db = require('./config/db'); // Importamos el pool de conexiones a MySQL
que configuramos en db.js

const app = express(); // Creamos la aplicación Express
const PORT = 3000; // Puerto en el que escuchará el servidor

// =====
// MIDDLEWARES
// =====
// Los middlewares son funciones que se ejecutan ANTES de llegar a las rutas.
// Se ejecutan en el orden en que se declaran.

app.use(cors()); // Permite que Vue (puerto 5173) haga peticiones a este
servidor (puerto 3000)
app.use(express.json()); // Permite que el servidor entienda cuerpos de peticiones
en formato JSON

// =====
// ENDPOINTS PARA CONTACTOS
// =====

// 1. Obtener todos los contactos (GET)
// Ruta: http://localhost:3000/api/contactos
app.get('/api/contactos', async (req, res) => {
  try {
    // db.query() ejecuta una consulta SQL y devuelve una promesa.
    // "rows" contiene el arreglo de resultados.
    const [rows] = await db.query('SELECT * FROM contactos ORDER BY id');

    // Respondemos con status 200 (OK) y el arreglo de contactos en formato JSON
    res.status(200).json({ exito: true, datos: rows });
  } catch (error) {
    // Si algo falla (ej: MySQL caído), capturamos el error y respondemos 500
    console.error('Error al obtener contactos:', error);
    res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
  }
});

// 2. Obtener un contacto por ID (GET)
// Ruta: http://localhost:3000/api/contactos/1
app.get('/api/contactos/:id', async (req, res) => {
  try {
    // req.params.id contiene el ID que viene en la URL.
    // Usamos "?" como placeholder para evitar Inyección SQL.
    const [rows] = await db.query('SELECT * FROM contactos WHERE id = ?',
[req.params.id]);

    // Si no se encontró ningún registro, devolvemos 404 (Not Found)
    if (rows.length === 0) {
      return res.status(404).json({ exito: false, mensaje: 'Contacto no encontrado'
});
    }
  }
}

```

```

    // Si existe, devolvemos el primer (y único) resultado
    res.status(200).json({ exito: true, datos: rows[0] });
  } catch (error) {
    console.error('Error al obtener contacto:', error);
    res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
  }
});

// 3. Crear un nuevo contacto (POST)
// Ruta: http://localhost:3000/api/contactos
app.post('/api/contactos', async (req, res) => {
  try {
    // Extraemos los campos del cuerpo JSON de la petición
    const { nombre, rfc, tipo, email, telefono } = req.body;

    // Validación de campos obligatorios (regla de negocio)
    if (!nombre || !rfc || !tipo) {
      return res.status(400).json({ exito: false, mensaje: 'Campos obligatorios: nombre, rfc, tipo' });
    }

    // INSERT: agregamos un nuevo registro a la tabla contactos.
    // Los "?" se reemplazan en orden con los valores del array.
    const [result] = await db.query(
      'INSERT INTO contactos (nombre, rfc, tipo, email, telefono) VALUES (?, ?, ?, ?, ?)',
      [nombre, rfc, tipo, email || null, telefono || null]
    );

    // result.insertId contiene el ID auto-generado del nuevo registro
    res.status(201).json({
      exito: true,
      mensaje: 'Contacto creado exitosamente',
      datos: { id: result.insertId, nombre, rfc, tipo }
    });
  } catch (error) {
    console.error('Error al crear contacto:', error);

    // Si el RFC ya existe, MySQL lanza el error 'ER_DUP_ENTRY'
    if (error.code === 'ER_DUP_ENTRY') {
      return res.status(400).json({ exito: false, mensaje: 'El RFC ya está registrado' });
    }

    res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
  }
});

// 4. Actualizar un contacto (PUT)
// Ruta: http://localhost:3000/api/contactos/1
app.put('/api/contactos/:id', async (req, res) => {
  try {
    const { nombre, rfc, tipo, email, telefono } = req.body;

```

```

// UPDATE: modificamos el registro cuyo id coincide con req.params.id
const [result] = await db.query(
  'UPDATE contactos SET nombre=?, rfc=?, tipo=?, email=?, telefono=? WHERE
id=?',
  [nombre, rfc, tipo, email, telefono, req.params.id]
);

// affectedRows indica cuántos registros fueron modificados.
// Si es 0, significa que el ID no existía.
if (result.affectedRows === 0) {
  return res.status(404).json({ exito: false, mensaje: 'Contacto no encontrado'
});
}

res.status(200).json({ exito: true, mensaje: 'Contacto actualizado', datos: {
id: parseInt(req.params.id), nombre, rfc, tipo } });
} catch (error) {
  console.error('Error al actualizar contacto:', error);
  res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
}
});

// 5. Eliminar un contacto (DELETE)
// Ruta: http://localhost:3000/api/contactos/1
app.delete('/api/contactos/:id', async (req, res) => {
  try {
    // DELETE: borra el registro con el ID especificado
    const [result] = await db.query('DELETE FROM contactos WHERE id = ?',
[req.params.id]);

    if (result.affectedRows === 0) {
      return res.status(404).json({ exito: false, mensaje: 'Contacto no encontrado'
});
    }

    res.status(200).json({ exito: true, mensaje: 'Contacto eliminado' });
  } catch (error) {
    console.error('Error al eliminar contacto:', error);
    res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
  }
});

// =====
// ENDPOINTS PARA MOVIMIENTOS (CAJA/BANCO)
// =====

// 1. Obtener todos los movimientos (GET)
// Ruta: http://localhost:3000/api/movimientos
app.get('/api/movimientos', async (req, res) => {
  try {
    // JOIN: combinamos movimientos con contactos para mostrar el nombre del
contacto
    // LEFT JOIN: incluye movimientos aunque no tengan contacto asociado
(contacto_id = NULL)

```

```

    const [rows] = await db.query(`
        SELECT m.*, c.nombre AS contacto_nombre
        FROM movimientos m
        LEFT JOIN contactos c ON m.contacto_id = c.id
        ORDER BY m.fecha DESC
    `);
    res.status(200).json({ exito: true, datos: rows });
} catch (error) {
    console.error('Error al obtener movimientos:', error);
    res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
}
});

// 2. Crear un nuevo movimiento (POST)
// Ruta: http://localhost:3000/api/movimientos
app.post('/api/movimientos', async (req, res) => {
    try {
        const { concepto, tipo, monto, fecha, contacto_id } = req.body;

        // Validaciones de negocio (reglas contables básicas)
        if (!concepto || !tipo || !monto || !fecha) {
            return res.status(400).json({ exito: false, mensaje: 'Campos obligatorios: concepto, tipo, monto, fecha' });
        }

        // El tipo solo puede ser 'Ingreso' o 'Egreso'
        if (!['Ingreso', 'Egreso'].includes(tipo)) {
            return res.status(400).json({ exito: false, mensaje: 'El tipo debe ser "Ingreso" o "Egreso"' });
        }

        // Convertimos el monto a número y verificamos que sea positivo
        const montoNumerico = parseFloat(monto);
        if (isNaN(montoNumerico) || montoNumerico <= 0) {
            return res.status(400).json({ exito: false, mensaje: 'El monto debe ser un número positivo' });
        }

        // INSERT del movimiento en la base de datos
        const [result] = await db.query(
            `INSERT INTO movimientos (concepto, tipo, monto, fecha, contacto_id) VALUES`
            + ` (?, ?, ?, ?, ?)`,
            [concepto, tipo, montoNumerico, fecha, contacto_id || null]
        );

        res.status(201).json({
            exito: true,
            mensaje: 'Movimiento registrado',
            datos: { id: result.insertId, concepto, tipo, monto: montoNumerico, fecha }
        });
    } catch (error) {
        console.error('Error al crear movimiento:', error);
        res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
    }
}

```

```

});

// 3. Generar reporte (resumen contable de tesorería) (GET)
// Ruta: http://localhost:3000/api/resumen
// Este endpoint calcula totales directamente en SQL usando SUM y CASE
app.get('/api/resumen', async (req, res) => {
  try {
    const [rows] = await db.query(`
      SELECT
        SUM(CASE WHEN tipo = 'Ingreso' THEN monto ELSE 0 END) AS totalIngresos,
        SUM(CASE WHEN tipo = 'Egreso' THEN monto ELSE 0 END) AS totalEgresos,
        COUNT(*) AS totalMovimientos
      FROM movimientos
    `);

    const resumen = rows[0];

    // Calculamos el saldo restando egresos de ingresos
    // Usamos "|| 0" para manejar el caso de que no haya movimientos (NULL)
    const saldo = (parseFloat(resumen.totalIngresos) || 0) -
      (parseFloat(resumen.totalEgresos) || 0);

    res.status(200).json({
      exito: true,
      datos: {
        totalIngresos: parseFloat(resumen.totalIngresos) || 0,
        totalEgresos: parseFloat(resumen.totalEgresos) || 0,
        saldo: saldo,
        totalMovimientos: parseInt(resumen.totalMovimientos)
      }
    });
  } catch (error) {
    console.error('Error al obtener resumen:', error);
    res.status(500).json({ exito: false, mensaje: 'Error interno del servidor' });
  }
});

// =====
// INICIAR SERVIDOR
// =====
// app.listen() pone al servidor a escuchar peticiones en el puerto especificado.
// El callback se ejecuta una vez que el servidor está listo.
app.listen(PORT, () => {
  console.log('');
  console.log('||  SERVIDOR ERP CON MySQL ACTIVO  ||');
  console.log('');
  console.log(` Puerto: http://localhost:${PORT}`);
  console.log(` Base de datos: erp_contable_jdc`); // ⚠ REEMPLAZA con tu BD
});

```

Paso 4.4 — Inicia el servidor

```
npm run dev
```

Deberías ver:

```
SERVIDOR ERP CON MySQL ACTIVO
```

```
Puerto: http://localhost:3000
```

```
Base de datos: erp_contable_jdc
```

✅ **Criterio de éxito:** El servidor inicia sin errores y los endpoints responden consultando MySQL.

● EJERCICIO 5: Verificación rápida en Postman (30 min)

Antes de terminar la sesión 1, verifica que los 3 endpoints principales funcionen con MySQL:

1. **GET** `http://localhost:3000/api/contactos` → debe devolver los 2 contactos de prueba (con tu nombre)
2. **POST** `http://localhost:3000/api/contactos` → crea un nuevo contacto
3. **GET** `http://localhost:3000/api/movimientos` → debe devolver los 2 movimientos de prueba (con tu cédula en el concepto)

Prueba de persistencia: Reinicia el servidor (`Ctrl+C` y luego `npm run dev`) y vuelve a hacer GET. Los datos **deben seguir ahí**. Esto confirma que ya no estás en memoria RAM.

📅 SESIÓN 2 — JUEVES 01/10/2026 (2 horas académicas)

🎯 **Objetivo de la sesión:** Documentar las 4 peticiones, validar precisión contable, responder preguntas reflexivas y realizar la **Prueba de Fuego**.

● EJERCICIO 6: Las 4 peticiones documentadas (45 min)

Según el contrato de aprendizaje, debes documentar estas **4 peticiones fundamentales** con sus respectivas respuestas del servidor.

✦ PETICIÓN 1: Crear cliente (POST)

Campo	Valor
Método	POST
URL	<code>http://localhost:3000/api/contactos</code>

Campo	Valor
Headers	Content-Type: application/json
Body	<pre>json { "nombre": "Distribuidora Los Andes C.A.", "rfc": "DLA2026100101", "tipo": "Cliente", "email": "ventas@andes.com", "telefono": "0412-1234567" }</pre>
Status esperado	201 Created

PETICIÓN 2: Registrar movimiento (POST)

Campo	Valor
Método	POST
URL	http://localhost:3000/api/movimientos
Headers	Content-Type: application/json
Body	<pre>json { "concepto": "Venta de servicios de consultoría", "tipo": "Ingreso", "monto": 2500.00, "fecha": "2026-10-01", "contacto_id": 3 }</pre>
Status esperado	201 Created

✦ PETICIÓN 3: Consultar datos (GET)

Campo	Valor
Método	GET
URL	http://localhost:3000/api/movimientos
Headers	(ninguno)
Status esperado	200 OK

PETICIÓN 4: Generar reporte (GET)

Campo	Valor
Método	GET
URL	http://localhost:3000/api/resumen
Headers	(ninguno)
Status esperado	200 OK

Para cada petición debes capturar:

- La configuración de la petición en Postman (URL, método, headers, body)

- La respuesta del servidor (status code + body JSON)
- Una captura de pantalla clara de ambas

● EJERCICIO 7: Validación de precisión contable (20 min)

Según la planificación, los reportes generados por el sistema deben **coincidir numéricamente** con cálculos hechos manualmente.

Paso 7.1 — Realiza estas operaciones en Postman (en orden):

1. POST movimiento: Ingreso de \$1,500.00 (concepto: "Venta inicial")
2. POST movimiento: Egreso de \$450.50 (concepto: "Compra insumos")
3. POST movimiento: Ingreso de \$2,500.00 (concepto: "Venta consultoría")
4. POST movimiento: Egreso de \$200.00 (concepto: "Servicios internet")

Paso 7.2 — Calcula manualmente:

Concepto	Cálculo manual
Total Ingresos	1500.00 + 2500.00 = \$4,000.00
Total Egresos	450.50 + 200.00 = \$650.50
Saldo	4000.00 – 650.50 = \$3,349.50
Total movimientos	4

Paso 7.3 — Consulta el reporte

GET <http://localhost:3000/api/resumen>

Paso 7.4 — Compara

Los valores devueltos por la API **deben coincidir exactamente** con tus cálculos manuales. Si hay discrepancia, revisa tu código.

✅ **Criterio de éxito:** Los valores coinciden. Documenta esta comparación en tu PDF.

🔥 EJERCICIO 8: LA PRUEBA DE FUEGO — Defensa en Vivo (25 min)

⚠️ **Esta es la parte más importante del taller.** El PDF representa el 50% de tu nota. El otro 50% es esta defensa en vivo. El docente pasará por tu puesto y te dará **3 minutos** para realizar cambios en tu código. **Debes documentar cómo lo resolviste en tu PDF.**

🔗 ¿Qué es la Prueba de Fuego?

Es una defensa técnica en vivo donde el docente te asignará **3 retos aleatorios** para verificar que realmente entiendes el código que entregaste. No se trata de memorizar, sino de demostrar que puedes **leer, modificar y debuggear** tu propio código bajo presión (como en un trabajo real).

📋 Los 3 Retos Posibles

El docente elegirá 3 de los siguientes retos. Debes completar **al menos 2** para aprobar.

Reto 1 — Cambio de Puerto:

"Apaga tu servidor. Cambia el puerto de escucha de 3000 a 3005 en tu código y haz que funcione."

Reto 2 — Extensión del Modelo:

"Agrega un campo nuevo llamado *telefono* a la tabla *contactos* en MySQL, actualiza tu *server.js* y haz que el endpoint POST lo reciba y lo guarde."

Reto 3 — Cambio de Configuración:

"Muéstrame en tu *db.js* cómo cambiarías la contraseña de la base de datos sin romper el servidor."

Reto 4 — Validación de Negocio:

"Agrega una validación en el endpoint POST de movimientos para que no se puedan registrar movimientos con concepto vacío. Muéstralo funcionando en Postman."

Reto 5 — Consulta con Filtro:

"Modifica el endpoint GET */api/movimientos* para que acepte un parámetro *?tipo=Ingreso* y solo devuelva los movimientos de ese tipo."

Cómo Documentar la Prueba de Fuego en tu PDF

En la **Sección 7 de tu PDF**, debes incluir un apartado llamado **"Prueba de Fuego - Defensa en Vivo"** con la siguiente estructura para **cada reto que te tocó**:

```
### Reto #X: [Nombre del reto]

**Enunciado del docente:**
"[Copia aquí el enunciado exacto que te dio el docente]"

**Pasos que realicé:**
1. [Describe el paso 1: qué archivo abriste, qué línea modificaste]
2. [Describe el paso 2]
3. [Describe el paso 3]

**Código modificado:**
``javascript
// Pega aquí el fragmento de código que modificaste
```

Resultado: [Describe qué pasó: ¿funcionó a la primera? ¿tuviste errores? ¿cómo los resolviste?]

Captura de pantalla: [Inserta captura de Postman o terminal mostrando el resultado exitoso]

¿Qué aprendí? [Reflexión de 3-4 líneas sobre qué concepto técnico reforzaste con este reto]

Reglas de la Prueba de Fuego

1. **Tiempo máximo:** 3 minutos por reto. El docente cronometrará.
2. **Puedes usar internet:** Si olvidas la sintaxis de `ALTER TABLE` o cómo se escribe `req.body.telefono`, búscalo. En la vida real, un desarrollador junior busca en Google, pero debe saber *qué* buscar.
3. **No puedes pedir ayuda al docente sobre sintaxis:** Solo puedes preguntar sobre el enunciado si no lo entendiste.
4. **Debes documentar TODO en el PDF:** Aunque no completes el reto en los 3 minutos, documenta hasta dónde llegaste y qué aprendiste.

⚠ Regla de Evaluación

- Si completas **2 o 3 retos** exitosamente → **Aprobado** (mantienes la nota del PDF)
- Si completas **0 o 1 reto** → **Nota del taller: 0%** (independientemente de lo perfecto que esté tu PDF)

Justificación pedagógica: Si no puedes defender y modificar tu código en vivo, se asume que no lo comprendes o que no es autoría propia. En el mundo laboral, un desarrollador que no puede modificar su propio código bajo revisión no es productivo.

● EJERCICIO 9: Elaboración del PDF de entrega (30 min)

Crea un documento PDF con la siguiente estructura exacta:

```
PORTADA
├─ Título: Documentación API REST - ERP Contable (Tesorería)
├─ Nombre y apellido del estudiante
├─ Cédula de identidad
├─ Carrera: Informática - 5to Semestre
├─ Unidad Curricular: Interfaces Web con el Usuario
├─ Docente: Ing. José Daniel Cadenas L.
├─ Fecha de entrega
└─ Semana 9

1. DIAGRAMA ENTIDAD-RELACIÓN
├─ Imagen del DER (DER_erp_contable.png)
└─ Descripción breve del modelo (entidades, atributos, relación 1:N)

2. DOCUMENTACIÓN DE LAS 4 PETICIONES
2.1 Crear cliente (POST /api/contactos)
    - Descripción del endpoint
    - Request (método, URL, headers, body)
    - Response (status code + body JSON)
    - Captura de pantalla de Postman
2.2 Registrar movimiento (POST /api/movimientos)
    [misma estructura]
2.3 Consultar datos (GET /api/movimientos)
    [misma estructura]
2.4 Generar reporte (GET /api/resumen)
    [misma estructura]

3. TABLA DE CÓDIGOS DE ESTADO HTTP
| Código | Significado | Uso en el ERP |
```

```
|-----|-----|-----|
| 200    | OK        | Consultas exitosas |
| 201    | Created   | Recursos creados   |
| 400    | Bad Request | Validaciones fallidas |
| 404    | Not Found  | Recursos no encontrados |
| 500    | Server Error | Errores internos |
```

4. VALIDACIÓN DE PRECISIÓN CONTABLE

- └─ Tabla de cálculos manuales
- └─ Respuesta del endpoint /api/resumen
- └─ Conclusión de la comparación

5. PREGUNTAS REFLEXIVAS

(Ver sección "Preguntas reflexivas" más abajo)

6. PRUEBA DE FUEGO - DEFENSA EN VIVO

- └─ Reto #1: [Nombre]
 - └─ Enunciado del docente
 - └─ Pasos que realicé
 - └─ Código modificado
 - └─ Resultado
 - └─ Captura de pantalla
 - └─ ¿Qué aprendí?
- └─ Reto #2: [Nombre]
 - └─ [misma estructura]
- └─ Reto #3: [Nombre] (si aplica)
 - └─ [misma estructura]

7. CONCLUSIONES

- └─ Reflexión personal (mínimo 1 párrafo)

Nombre del archivo: Apellido_Nombre_Semana9_API_REST.pdf

? PREGUNTAS REFLEXIVAS (Sección 5 del PDF)

Responde estas preguntas en tu documento PDF. Cada respuesta debe tener **mínimo 5 líneas** y demostrar comprensión conceptual.

Pregunta 1 — Sobre el DER

Explica la relación entre las entidades **contactos y **movimientos**. ¿Por qué elegiste cardinalidad 1:N y no N:M? ¿Qué pasaría si un movimiento pudiera tener múltiples contactos? ¿Cómo cambiaría el modelo de datos?**

Pregunta 2 — Sobre persistencia

Compara el comportamiento de tu servidor en la Semana 7-8 (datos en memoria RAM) con el de la Semana 9 (datos en MySQL). ¿Qué ventajas reales observaste al migrar? Menciona al menos 3 ventajas concretas.

Pregunta 3 — Sobre códigos HTTP

Si un estudiante intenta crear un movimiento con monto negativo, ¿qué código HTTP debería recibir y por qué? Explica la diferencia entre un error 400 (Bad Request) y un error 500 (Server Error) en el contexto de este ERP.

Pregunta 4 — Sobre validaciones

¿Por qué es importante validar los datos en el backend (Express) y no solo en el frontend (Vue)? Da un ejemplo concreto de qué pasaría si un usuario malintencionado envía datos inválidos directamente a la API saltándose el frontend.

Pregunta 5 — Sobre precisión contable

En el Ejercicio 7 comparaste cálculos manuales con la respuesta del endpoint `/api/resumen`. ¿Por qué es crítica esta validación en un sistema financiero? ¿Qué consecuencias tendría un error de redondeo o cálculo en un sistema real de tesorería?

RÚBRICA DE EVALUACIÓN (10% Sumativa - Individual)

Criterio	Excelente (100%)	Satisfactorio (70%)	En Desarrollo (40%)	No Evidenciado (0%)	Peso
1. Diagrama Entidad-Relación	DER completo con entidades <code>contactos</code> y <code>movimientos</code> , atributos, PK, FK, cardinalidad 1:N correcta y notación estándar.	DER presente con entidades y atributos, pero con errores menores en cardinalidad o faltan algunas FK.	DER incompleto, solo 1 entidad o con errores significativos en el modelado.	No presenta DER.	2.0%
2. Implementación MySQL	Base de datos creada con tablas, tipos de datos apropiados, restricciones (UNIQUE, CHECK, FK). Servidor Express usa MySQL correctamente y los datos persisten al reiniciar.	Base de datos funcional pero con errores menores en tipos de datos o faltan algunas restricciones.	Base de datos creada pero el servidor aún usa memoria RAM o tiene errores de conexión.	No implementa MySQL.	2.0%
3. Documentación de 4 peticiones	Documenta las 4 peticiones (crear cliente, registrar movimiento, consultar, generar	Documenta las 4 peticiones pero faltan detalles en request/response	Documenta menos de 4 peticiones o la documentación es superficial.	No documenta las peticiones.	2.5%

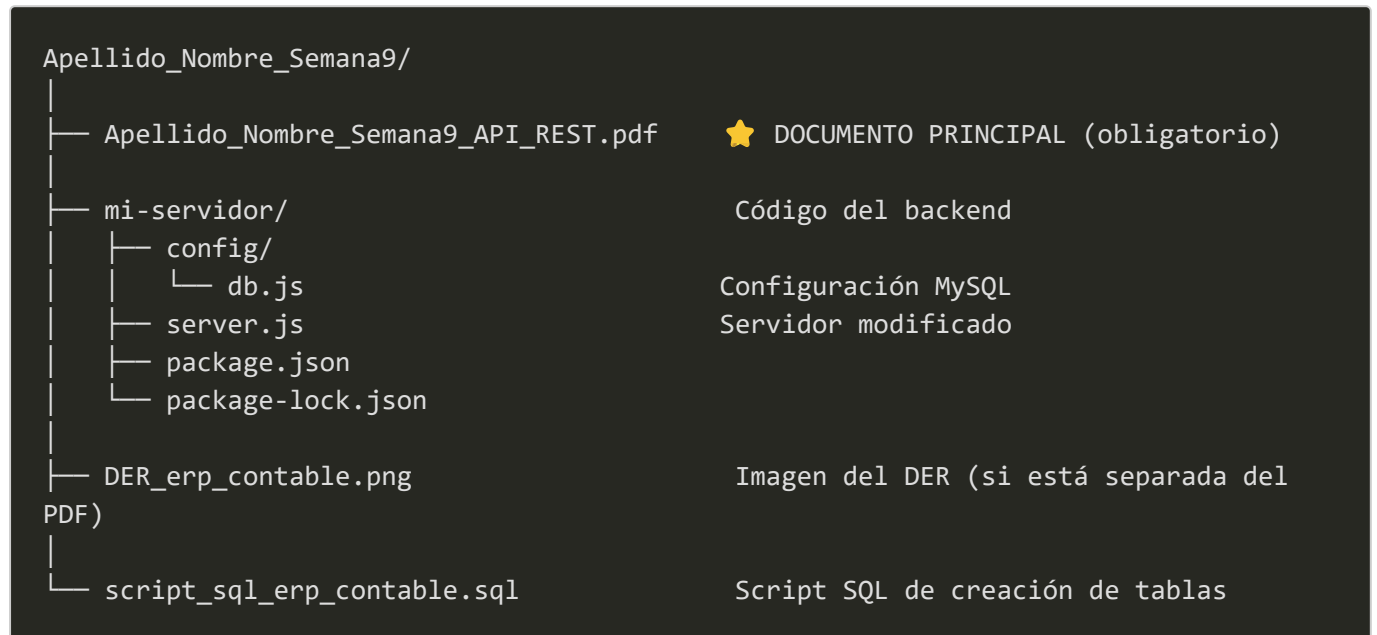
Criterio	Excelente (100%)	Satisfactorio (70%)	En Desarrollo (40%)	No Evidenciado (0%)	Peso
	reporte) con request/response completos, códigos de estado y capturas claras de Postman.	o las capturas no son claras.			
4. Interpretación de códigos HTTP	Explica correctamente los códigos de estado (200, 201, 400, 404, 500) y los relaciona con cada respuesta del servidor en la documentación.	Menciona los códigos de estado pero sin explicación detallada o no los relaciona con las respuestas.	Solo lista los códigos sin contexto o con errores en la interpretación.	No interpreta códigos de estado.	1.0%
5. Validación de precisión y calidad del PDF	Los cálculos manuales coinciden con la respuesta del endpoint /api/resumen . PDF profesional con estructura clara, portada, preguntas reflexivas respondidas y conclusiones. Sin errores ortográficos.	Coincidencia parcial o PDF con estructura aceptable pero errores menores de formato.	No realiza la validación de precisión o PDF desorganizado.	No entrega PDF o no realiza la validación.	1.0%
6. Prueba de Fuego (Defensa en Vivo)	Completa 2 o 3 retos exitosamente en los 3 minutos. Documenta en el PDF los pasos, código modificado, resultado y aprendizaje de cada reto con capturas de pantalla.	Completa 2 retos pero la documentación en el PDF es incompleta (falta código o captura).	Completa solo 1 reto o la documentación es superficial.	No completa ningún reto o no documenta la Prueba de Fuego en el PDF.	1.5%

Total: 10%

INSTRUCCIONES DE ENTREGA

Estructura de la carpeta en Google Drive

Sube una carpeta con la siguiente estructura exacta a la carpeta compartida de Drive que el docente proporcionará:



Formato del PDF

El PDF debe incluir **obligatoriamente**:

1. **Portada** con datos completos del estudiante
2. **Diagrama Entidad-Relación** (imagen clara y legible)
3. **Documentación de las 4 peticiones** (cada una con request, response, código de estado y captura de Postman)
4. **Tabla de códigos de estado HTTP**
5. **Validación de precisión contable** (cálculos manuales vs respuesta del endpoint)
6. **5 preguntas reflexivas** respondidas
7. **Prueba de Fuego documentada** (con pasos, código, resultado, captura y aprendizaje por cada reto)
8. **Conclusiones** (mínimo 1 párrafo)

Fecha y forma de entrega

- **Fecha límite:** Jueves 01/10/2026, al finalizar la sesión de clase
- **Forma:** Carpeta comprimida (.zip) o carpeta con la estructura indicada, subida a la carpeta de Google Drive compartida por el docente
- **Nombre del PDF:** `Apellido_Nombre_Semana9_API_REST.pdf`

Ten este documento a la mano mientras realizas los ejercicios.

1. ¿Por qué MySQL y no seguir en memoria RAM?

Característica	Memoria RAM (Semana 7-8)	MySQL (Semana 9)
Persistencia	Se pierde al reiniciar	Permanece entre sesiones
Concurrencia	Solo un usuario	Múltiples usuarios simultáneos
Volumen	Limitada por memoria del servidor	Maneja millones de registros
Integridad	Sin validaciones de estructura	Restricciones (FK, UNIQUE, CHECK)
Consultas complejas	Arreglos en JavaScript	SQL con JOIN, GROUP BY, SUM

2. Tipos de datos MySQL más usados en este proyecto

Tipo	Uso	Ejemplo
INT AUTO_INCREMENT	Llaves primarias	id
VARCHAR(n)	Textos cortos	nombre VARCHAR(100)
DECIMAL(10,2)	Montos de dinero	monto DECIMAL(10,2) → 99999999.99
ENUM('A', 'B')	Valores fijos	tipo ENUM('Ingreso', 'Egreso')
DATE	Solo fecha	fecha DATE → '2026-10-01'
TIMESTAMP	Fecha y hora automática	fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP

3. Foreign Keys y cardinalidad

Foreign Key (FK): Campo que referencia la PK de otra tabla. Garantiza integridad referencial.

```
FOREIGN KEY (contacto_id) REFERENCES contactos(id) ON DELETE SET NULL
```

- contacto_id en movimientos apunta a id en contactos
- ON DELETE SET NULL: si se elimina un contacto, sus movimientos no se borran, solo queda contacto_id = NULL

Cardinalidad 1:N: Un contacto puede tener **muchos** movimientos, pero un movimiento pertenece a **un solo** contacto.

4. Códigos de estado HTTP en este proyecto

Código	Nombre	Cuándo usarlo en el ERP
200	OK	GET y PUT exitosos

Código	Nombre	Cuándo usarlo en el ERP
201	Created	POST exitoso (recurso creado)
400	Bad Request	Validación fallida (campos vacíos, monto negativo, RFC duplicado)
404	Not Found	ID no existe en la BD
500	Server Error	Error de conexión a MySQL, bug en el código

5. Diferencia entre `db.query()` con arreglos en memoria

Antes (memoria RAM):

```
let contactos = [];
app.get('/api/contactos', (req, res) => {
  res.json(contactos); // Devuelve el arreglo en memoria
});
```

Ahora (MySQL):

```
app.get('/api/contactos', async (req, res) => {
  const [rows] = await db.query('SELECT * FROM contactos');
  res.json(rows); // Devuelve resultado de la consulta SQL
});
```

6. Errores comunes y soluciones

Error	Causa	Solución
ER_ACCESS_DENIED_ERROR	Credenciales incorrectas en <code>db.js</code>	Verifica usuario, password y permisos
ER_DUP_ENTRY	RFC duplicado	Usa un RFC único o elimina el registro duplicado
ECONNREFUSED	MySQL no está corriendo	Inicia el servicio MySQL
Los datos no persisten	El servidor aún usa arreglos	Verifica que uses <code>db.query()</code> en todos los endpoints
Cannot POST /api/...	Falta <code>express.json()</code>	Agrega <code>app.use(express.json())</code> antes de las rutas

7. Comandos MySQL útiles

```
SHOW DATABASES;      -- Ver bases de datos
USE erp_contable_jdc; -- Seleccionar BD (reemplaza con tu BD)
SHOW TABLES;        -- Ver tablas
DESCRIBE contactos;   -- Ver estructura de tabla
```

```
SELECT * FROM contactos;           -- Ver todos los registros
ALTER TABLE contactos ADD telefono VARCHAR(20); -- Agregar columna (Reto Prueba de Fuego #2)
DELETE FROM contactos WHERE id=1;  -- Eliminar registro específico
DROP DATABASE erp_contable_jdc;    -- ⚠ Eliminar BD completa (cuidado)
```

8. Tips para la Prueba de Fuego

- **Respira:** 3 minutos es tiempo suficiente si sabes qué buscar.
- **Lee el error:** La consola de Node.js te dice exactamente en qué línea falló.
- **Usa internet:** Si olvidas la sintaxis de `ALTER TABLE`, búscala. Lo importante es saber *qué* buscar.
- **Prueba en Postman:** Después de cada modificación, verifica en Postman que el endpoint funcione.
- **No entres en pánico con el Reto 3:** Cambiar la contraseña en `db.js` es solo editar un string. No hay que reiniciar MySQL ni hacer nada complejo.

✅ CHECKLIST FINAL

Antes de entregar, verifica:

- ☐ MySQL está instalado y corriendo
- ☐ Base de datos `erp_contable_[tus iniciales]` creada con usuario `erp_user`
- ☐ Tablas `contactos` y `movimientos` creadas con estructura correcta
- ☐ Diagrama Entidad-Relación creado y exportado como imagen
- ☐ Servidor Express modificado para usar MySQL (no memoria RAM)
- ☐ Los datos persisten al reiniciar el servidor
- ☐ Las 4 peticiones funcionan correctamente en Postman
- ☐ Capturas de pantalla claras de cada petición
- ☐ Validación de precisión contable realizada y documentada
- ☐ PDF con portada, DER, 4 peticiones, tabla de códigos, preguntas reflexivas, **Prueba de Fuego documentada** y conclusiones
- ☐ Carpeta subida a Google Drive con la estructura correcta

🎯 OBJETIVO DE APRENDIZAJE ALCANZADO

Al completar este taller, el estudiante habrá demostrado:

- ☒ Modelar entidades de un sistema de tesorería usando DER
- ☒ Configurar MySQL como sistema de persistencia para el backend
- ☒ Implementar operaciones CRUD reales sobre base de datos relacional
- ☒ Documentar profesionalmente una API REST con evidencia de peticiones
- ☒ Interpretar códigos de estado HTTP y formatos JSON
- ☒ Validar la precisión de los cálculos financieros del sistema
- ☒ **Defender y modificar su código en vivo** (Prueba de Fuego), demostrando comprensión real de la arquitectura
- ☒ Migrar exitosamente de memoria RAM a base de datos real

¡Éxito en tu práctica de la Semana 9!

Documento elaborado para la Unidad Curricular Interfaces Web con el Usuario (INU-554), Carrera de Informática, 5to Semestre. Docente: Ing. José Daniel Cadenas L. Septiembre 2026.